

## My learnings

• use `display(df.head())` along with `df.shape` as it will not override by `df.shape`

• To print all columns use `print(df.columns)`

It give u output like this

```
• Index(['Player Name', 'Team', 'Runs', 'Matches', 'Inn', 'No', 'HS', 'AVG', 'BF', 'SR', '100s', '50s', '4s', '6s'],
      dtype='object')
```

• But to just get the names of columns , you can use

```
1 for col in df.columns:
2     print(col)

... Player Name
Team
Runs
Matches
Inn
No
HS
AVG
BF
SR
100s
50s
4s
6s
```

• Here `df.columns.tolist()` is a list of columns in dictionary

• `df.columns` only give something like first image , to get in list we use `.tolist()` function

### What is an "Object" in Pandas?

Think of an "object" simply as the computer's word for **Text** or a **Mix of different things**.

In Pandas, if a column is filled with pure numbers (like `15` , `20` , `100` ), the computer recognizes them as numbers. It calls them `int64` (whole numbers) or `float64` (decimals). It knows it can do math with these, like adding or finding an average.

However, if a column contains words, letters, or special symbols, the computer treats it as plain text. It calls this text an `object` .

### The `.str` Magic Wand

This is exactly where `.str` comes to the rescue!

Think of `.str` as a bridge or a magic wand. When you add `.str` after your column name, you are giving Pandas a very specific command.

You are telling it: "Open up this column, treat every single row inside it as a regular piece of text, and get ready to apply a text tool to all of them at the exact same time."

Once you use `.str`, the column temporarily acts like normal text, unlocking all the cool text features.

Python



```
df['Player Name'].str.split(' ')
```

#### 4. Replacing Symbols (What we did!)

If you want to look at every single score and delete the star symbol from the text:

Python



```
df['HS'].str.replace('*', '')
```

In short, whenever you need to clean, edit, or search through a column full of text, you will always type `.str` first!

.SO object has text inside to use text properties like upper,lower,replace,etc we use `.str` that takes whole column text and do the thing for us and yes it takes whole column text no each row at a time

.Also I done mistake I wrote :

`df['HS'] = df['HS'].sort_values` , this will work but not work means it sort the values but values also have indexes so can't be sorted as indexes are not sorted they are at fixed position so though index one has highest value it still be at one , as indexs are not sorted.

We have to sort the whole table and to do that we use `df.sort_values(by = "HS")` , this means sort whole dataframe according to HS column.

When you use `.str.replace()`, Pandas is extremely strict. It demands that both the thing you are looking for AND the thing you are replacing it with are plain text.

When you want to replace an exact cell value with a special missing object like None, you should use the normal Pandas replace tool.

